

NAZARBAYEV
UNIVERSITY

Programming for Engineers

Week 12: Searching and Sorting Algorithms

Yanwei Wang
School of Engineering
Nazarbayev University

Fall 2026





Week 12 at a glance

Lecture 34

Linear and brute-force search

- State the required output.
- Search values and records.
- Enumerate candidates.

Lecture 35

Bisection search

- Justify interval reduction.
- Test search boundaries.
- Locate matches and intervals.

Lecture 36

Sorting and ranking

- Trace sorting algorithms.
- Preserve complete records.
- Compare methods and test.

Specify the result $\rightarrow$ Search deliberately
$\rightarrow$ Reorder with meaning $\rightarrow$ Verify the evidence

Three approximately 50-minute teaching units; optional reference material follows the core lectures.

Course scope and continuity

Official Session 12

- Search algorithms and brute-force search.
- Linear search and bisection search.
- Sorting algorithms.
- Assignment 7 preparation.

Build on earlier weeks

- Functions, lists, dictionaries, and objects.
- Loop invariants and recursion.
- Tests, exceptions, and preservation of data.
- Time, space, and preparation costs.

Teaching scope

Small transparent implementations first; documented library tools for application code. Numeric examples and limits are illustrative, not certified design rules.

[C] ENG 101 Course Specification, Session 12; CLOs 1 and 2.

NAZARBAYEV UNIVERSITY

LECTURE 34

Linear and Brute-Force Search

Specify the result, inspect candidates, and preserve engineering meaning

NAZARBAYEV UNIVERSITY

Lecture 34 learning goals

1. Distinguish membership, first match, all matches, and best candidate.
2. Implement and trace a linear search that returns an index or `None`.
3. Search conditions, names, and complete engineering records.
4. Enumerate discrete candidates without skipping or duplicating cases.
5. Test boundary cases and justify the amount of work.

Central question

What must the caller receive, and what does that result mean?

[W] Sections 10.2–10.8; teaching adaptation.

NAZARBAYEV UNIVERSITY

Warm-up: three questions, one dataset

```
# Pressures in acquisition order, in kPa.
values = [95, 108, 99, 115, 102]
# A: Is 115 present?
# B: At which index does 115 first occur?
# C: Which readings exceed 100 kPa?
```

Predict the returned information

- A Boolean?
- One position?
- Several index–value pairs?

Individual then pairs - 3 min

[W] Sections 10.2–10.8; teaching adaptation.

NAZARBAYEV UNIVERSITY

A search begins with a contract

Question	Returned result	Absence
Does a match exist?	`True` or `False`	`False`
Where is the first match?	An integer index	`None`
Which records match?	A list of matching records	An empty list
Which is best under a rule?	A selected record	`None`

Also state

Comparison rule, units, ordering, duplicates, and whether the input may change.

[W] Sections 10.2–10.8; teaching adaptation.

NAZARBAYEV UNIVERSITY

Linear search: check one candidate at a time

1. Start at the first element.
2. Test the requested equality or condition.
3. Return immediately when the required first match is found.
4. Otherwise advance to the next element.
5. Report absence only after the whole sequence has been examined.

Why it works

Before index i is tested, every earlier element has failed the same test.

[W] Sections 10.2–10.8; teaching adaptation.

NAZARBAYEV UNIVERSITY

A reusable first exact-match search

```
def linear_search(values, target):
    """Return the FIRST exact-match index, or None; preserve input order."""
    for index, value in enumerate(values):
        # Earlier positions have already failed the equality test.
        if value == target:
            return index
    # Absence is established only after every candidate has been checked.
    return None
```

Contract

Return the first index where `value == target`; return `None` when absent. Do not mutate the input.

[W] Sections 10.2–10.8; teaching adaptation.

NAZARBAYEV UNIVERSITY

Trace every equality test

Index	Value (kPa)	Equal to 115?	Action
0	95	False	continue
1	108	False	continue
2	99	False	continue
3	115	True	return 3

Not visited

Index 4 is not needed once the first exact match is established.

[W] Sections 10.2–10.8; teaching adaptation.

Index zero is a valid result

```
index = linear_search([95, 108, 99], 95)
print('Index:', index)
print('Explicit found test:', index is not None)
print('Incorrect truthiness test:', bool(index))
```

```
Index: 0
Explicit found test: True
Incorrect truthiness test: False
```

Use an explicit absence check

`if index is not None:` is correct for this index-or-None contract.

[W] Sections 10.2–10.8; teaching adaptation.

NAZARBAYEV UNIVERSITY

Test endpoints, duplicates, and absence

Input	Target	Expected result
[]	7	None
[7]	7	0
[7]	8	None
[5, 7, 7]	7	1
[5, 7, 9]	9	2

A first-match promise

With duplicates, returning a later matching index would violate this particular contract.

[W] Sections 10.2–10.8; teaching adaptation.

NAZARBAYEV UNIVERSITY

Turn predictions into regression checks

```
from week12_algorithms import linear_search
# Independent expected indices include empty, endpoint, and duplicate cases.
cases = [([], 7, None), ([7], 7, 0), ([7], 8, None),
         ([5, 7, 7], 7, 1), ([5, 7, 9], 9, 2)]
for values, target, expected in cases:
    assert linear_search(values, target) == expected
print('Five linear-search cases passed')
```

Run, modify one expected answer, and explain the failure

[W] Sections 10.2–10.8; teaching adaptation.

NAZARBAYEV UNIVERSITY

Search for a condition, not just an exact value

```
def first_above(readings, limit):
    """Return the first index strictly above limit,
     or None; units must agree."""
    for index, reading in enumerate(readings):
        # Use the original measurement order, not a
     value-sorted copy.
        if reading > limit:
            return index
    return None
```

Temperature record

`[42.1, 45.3, 49.8, 53.2, 58.7, 62.4, 66.1]`

Find the first reading strictly above $60.0\,^{\circ}\mathrm{C}$.

Boundary rule

A reading equal to the limit is not above it.

[W] Sections 10.2–10.8; teaching adaptation.

NAZARBAYEV UNIVERSITY

A position is not automatically a time

```
readings = [42.1, 45.3, 49.8, 53.2, 58.7, 62.4,
     66.1]
index = first_above(readings, 60.0)
if index is not None:
    time_s = 2.0 * index  # Index is not itself a
     physical time.
    print(f'Index {index}: {readings[index]:.1f} deg
      C at {time_s:.1f} s')
else:
    print('No reading above the stated limit')
```

[W] Section 10.3; illustrative sampling interval added explicitly.

Declared sampling metadata

$t_i = t_0 + i\Delta t$, with $t_0 = 0$ s and $\Delta t = 2$ s in this illustration.

Independent check

Index 5 corresponds to 10 s, not 5 s.

NAZARBAYEV UNIVERSITY

Do not sort away the question you were asked

acquisition	75	95	70	85
value-sorted	70	75	85	95

First reading above 80

In acquisition order: index 1, value 95.
In sorted order: index 2, value 85. These answer different questions.

[W] Sections 10.2–10.8; teaching adaptation.

NAZARBAYEV UNIVERSITY

All matches require a complete scan

```
def all_above(readings, limit):
    """Return all (index, value) matches, retaining
     acquisition order."""
    matches = []
    for index, reading in enumerate(readings):
        if reading > limit:
            # Keep the measurement's location
     attached to its value.
            matches.append((index, reading))
    return matches
```

Output for the pressure example

`[(1,108), (3,115), (4,102)]`
Keep both location and value.

Cost model

n tests; k stored matches.
Time $\Theta(n)$, output space $\Theta(k)$.

[W] Sections 10.2–10.8; teaching adaptation.

Names and identifiers need explicit rules

```
materials = [{'name': 'steel', 'density_kg_m3':
    7850},
             {'name': 'aluminum', 'density_kg_m3':
    2700}]
found = find_material(materials, ' Steel ')
if found is not None:
    print(found['name'], found['density_kg_m3'], 'kg
    /m^3')
print('Missing:', find_material(materials, 'copper')
    )
```

This material-name policy

" Steel " and "steel" denote the same material name.

Not a universal text rule

Asset IDs P-007 and P-7 may be different. Preserve identity.

[W] Sections 10.2–10.8; teaching adaptation.

NAZARBAYEV UNIVERSITY

Return the record without losing metadata

```
def find_material(records, query):
    """First named material record or None; names are case-insensitive here."""
    # Normalize material names only; do not strip zeros from asset IDs.
    wanted = query.strip().lower()
    for record in records:
        if record["name"].strip().lower() == wanted:
            return record  # This is a reference, not an independent copy.
    return None
```

A found dictionary remains linked to the input

Changing its fields would also change that record in the original list. Copy deliberately when independent edits are required.

[W] Sections 10.2–10.8; teaching adaptation.

NAZARBAYEV UNIVERSITY

Near a target is a different predicate

```python
def first_close(values, target, tolerance):
    """First match within absolute tolerance, not
     necessarily the nearest.
    Assume finite numeric inputs in one unit and
     tolerance >= 0.
    """
    for index, value in enumerate(values):
        # Tolerance has the same physical unit as value
     and target.
        if abs(value - target) <= tolerance:
            return index
    return None
```

Absolute tolerance

$|v - v_{\text{target}}| \leq \varepsilon$.
The tolerance has the same unit as the values.

Precondition

Finite compatible values and $\varepsilon \geq 0$; choose tolerance from the task.

[W] Sections 10.2–10.8; teaching adaptation.

NAZARBAYEV UNIVERSITY

Prediction: first close or nearest?

```
# Voltages and an absolute tolerance of 0.1 V.
values_v = [9.91, 10.02, 10.00]
index = first_close(values_v, 10.0, 0.1)
```

Answer without execution

Which index is returned? Which index has the smallest absolute error?

Pairs - 3 min

Modify the specification

What must change if the nearest reading is required instead?

[W] Sections 10.2–10.8; teaching adaptation.

NAZARBAYEV UNIVERSITY

Brute force can be the right engineering method

Candidate space

Try explicitly listed catalog options or combinations. Linear search is one simple brute-force pattern.

Useful when

- The candidate space is small.
- Each test is understandable.
- A complete search is affordable.

Specify before searching

- Which candidates are admissible?
- What counts as feasible?
- First, all, or best?

[W] Sections 10.2–10.8; teaching adaptation.

NAZARBAYEV UNIVERSITY

A finite area search with visible units

```
from week12_algorithms import first_feasible_area
# Course-book example: uniform axial stress; ascending catalog in m^2.
areas_m2 = [50e-6, 75e-6, 100e-6, 125e-6, 150e-6]
area = first_feasible_area(areas_m2, 12000.0, 80e6)
if area is not None:
    print(f'Area = {area * 1e6:.0f} mm^2')
    print(f'Stress = {12000.0 / area / 1e6:.1f} MPa')
```

Why the first feasible area is smallest

The supplied catalog is ascending. That conclusion would not follow from arbitrary input order.

[W] Section 10.4; catalog and numerical values retained.

NAZARBAYEV UNIVERSITY

An original combination-search exercise

Ideal series-resistor catalog

Four entries: $[2, 4, 8, 10]\,\Omega$. Find every pair of distinct entries with total resistance $12\,\Omega$.

Avoid duplicates

Inspect only pairs $i < j$.
Do not reuse one physical entry twice.

Predict and enumerate - 3 min

Prediction

Which two pairs should be returned?
How many candidate pairs exist?

Original teaching exercise within Session 12; assumptions stated on slide.

NAZARBAYEV UNIVERSITY

Enumerate each distinct pair once

```python
def resistor_pairs(resistances_ohm, target_ohm):
    """All i<j exact-sum pairs from a catalog of ideal integer-ohm values."""
    matches = []
    for i in range(len(resistances_ohm)):
        for j in range(i + 1, len(resistances_ohm)):
            # Distinct entries; each unordered pair is tested once.
            if resistances_ohm[i] + resistances_ohm[j] == target_ohm:
                matches.append((i, j))
    return matches
```

Count candidate evaluations

$\binom{n}{2} = n(n-1)/2$. If every matching pair is required, inspect every candidate pair.

Original teaching exercise within Session 12; assumptions stated on slide.

NAZARBAYEV UNIVERSITY

First feasible does not mean lowest cost

```
def cheapest_feasible(records, required_fos=2.0):
    """Return lowest-cost feasible record, or None; keep
     first equal-cost tie.
    Assume records have finite numeric cost_USD and fos
     fields.
    """
    best = None
    for record in records:
        if record["fos"] >= required_fos:
            # Do not stop at the first feasible record:
     later cost may be lower.
            if best is None or record["cost_USD"] < best[
     "cost_USD"]:
                best = record
    return best
```

[W] Section 10.7; original record values retained.

Book example

Feasible designs in order: B (150), C (180), D (130), costs in USD.

Answer

D has the lowest listed cost. Keep searching after B.

NAZARBAYEV UNIVERSITY

Debugging clinic: absence reported too early

```
# INTENTIONAL LOGIC DEFECT: absence is declared after
      only one failed test.
def broken_search(values, target):
    for i, value in enumerate(values):
        if value == target:
            return i
        return None  # Repair: move this after the
     loop.
print(broken_search([2, 4, 6], 6))  # Wrong: expected
      index 2.
```

Explain before editing

1. Which indices are visited?
2. What should the result be?
3. Where does `return None` belong?

Pairs - 3 min

[W] Sections 10.2–10.8; teaching adaptation.

NAZARBAYEV UNIVERSITY

A compact search review checklist

1. State the output: membership, index, records, or optimum.
2. State units, order, equality/tolerance, and duplicate rules.
3. Inspect every candidate that the contract requires.
4. Separate not found from valid index zero.
5. Check empty, first, last, duplicate, and absent cases.
6. Count candidate evaluations and preserve raw data.

[W] Sections 10.2–10.8; teaching adaptation.

NAZARBAYEV UNIVERSITY

Lecture 34 exit ticket

One small task

For `[20, 35, 30, 40]`, find the first value strictly above 30. Return an index or `None`.

1. Give the answer and the number of comparisons.
2. Give an input that returns index zero.
3. What changes if all matching values are required?
4. Why is sorting first unnecessary and potentially misleading?

Individual - 3 min

[W] Sections 10.2–10.8; teaching adaptation.

NAZARBAYEV UNIVERSITY

LECTURE 35

Bisection Search in Ordered Data

Keep a valid candidate interval and discard only what the ordering justifies

NAZARBAYEV UNIVERSITY

Lecture 35 learning goals

1. Distinguish list bisection search from numerical root finding.
2. State the sortedness and comparison assumptions.
3. Trace and implement an exact binary search with shrinking bounds.
4. Distinguish any match, first match, and an insertion position.
5. Locate a calibration interval with explicit boundary behavior.
6. Include preparation cost and original-index meaning.

Central question

Why is it valid to discard these candidates?

[W] Sections 10.9–10.12; [G] Section 10.1.2; teaching adaptation.

NAZARBAYEV UNIVERSITY

Warm-up: order provides information

voltage (V)	0	0.5	1.0	1.5	2.0	2.5	3.0

Target: 2.5 V

The middle value is 1.5 V. Which part can no longer contain an exact match?

Justify the elimination

Use the ascending-order assumption, not a guess about where the target looks likely to be.

Pairs - 2 min

[W] Sections 10.9–10.12; [G] Section 10.1.2; teaching adaptation.

NAZARBAYEV UNIVERSITY

Same halving idea; different search problem

Aspect	Week 3 root finding	Week 12 list search
Unknown	A continuous input x	A stored element index
Evidence	Continuity and a valid root bracket	Ascending, comparable stored values
Success	A stated approximation criterion	An exact stored match
Failure/stop	Tolerance or work-limit report	Empty candidate interval

Do not import the wrong assumption

List search does not require a sign change. A numerical root need not appear as an element in a list.

[W] Sections 10.9–10.12; [G] Section 10.1.2; teaching adaptation.

NAZARBAYEV UNIVERSITY

Preconditions are part of the algorithm

This implementation assumes

- Ascending sequence.
- Consistent equality and ordering.
- Random access by integer index.
- No concurrent changes during search.

Engineering preparation

- Convert compared quantities to one unit.
- Deal explicitly with missing or NaN data.
- Use the same key for sorting and searching.

Cost qualification

The $O(\log n)$ search claim excludes a full sortedness validation on every call.

[W] Sections 10.10 and 10.31; [B] bisect notes; technical assumptions made explicit.

Define the exact-match contract

Input

An existing ascending list of consistently comparable values and a target.

Output

An index whose value equals the target, or `None` if no exact match exists.

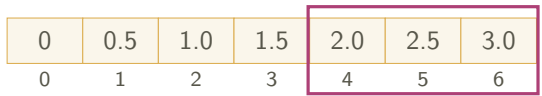
Duplicate policy

This first version returns **any** matching index. It does not promise the first one.

[W] Sections 10.9–10.12; [G] Section 10.1.2; teaching adaptation.

NAZARBAYEV UNIVERSITY

The candidate interval is inclusive



Invariant

If a match exists, at least one matching index remains between `low` and `high`, inclusive.

[W] Sections 10.9–10.12; [G] Section 10.1.2; teaching adaptation.

NAZARBAYEV UNIVERSITY

Complete exact binary-search implementation

```
def binary_search(values, target):
    """ANY exact-match index, or None; values must be ascending and ordered."""
    low = 0
    high = len(values) - 1
    # If target exists, at least one match remains inside [low, high].
    while low <= high:
        mid = (low + high) // 2
        value = values[mid]
        if value == target:
            return mid
        if value < target:
            low = mid + 1  # Exclude the inspected midpoint.
        else:
            high = mid - 1
    return None  # The candidate interval is empty.
```

[W] Sections 10.9–10.12; [G] Section 10.1.2; teaching adaptation.

NAZARBAYEV UNIVERSITY

Successful trace: target 2.5 V

Step	low	high	mid	value	Action
1	0	6	3	1.5	low $= 4$
2	4	6	5	2.5	return 5

Check the evidence

Two midpoint probes; the returned index points to the stored target. The data remain unchanged.

[W] Sections 10.9–10.12; [G] Section 10.1.2; teaching adaptation.

NAZARBAYEV UNIVERSITY

An absent value is a successful search outcome

Step	low	high	mid	value	Action
1	0	6	3	1.5	low $= 4$
2	4	6	5	2.5	high $= 4$
3	4	4	4	2.0	high $= 3$

Target: 1.7 V

Now low $= 4$ exceeds high $= 3$. Return `None`; do not return the last midpoint.

[W] Sections 10.9–10.12; [G] Section 10.1.2; teaching adaptation.

NAZARBAYEV UNIVERSITY

Why the updates use mid + 1 and mid - 1

```
# INTENTIONALLY WRONG update.
# With values=[1, 3], target=2:
low = 0
high = 1
mid = (low + high) // 2
low = mid  # Still zero: no progress.
```

Repair the reasoning

The midpoint was tested and failed equality. Keeping it can reproduce the same interval.

Progress measure

Number of remaining candidates: $\max(0, high - low + 1)$. It decreases on every non-returning pass.

Trace two passes - 3 min

[W] Sections 10.9–10.12; [G] Section 10.1.2; teaching adaptation.

NAZARBAYEV UNIVERSITY

The smallest cases catch the boundary bugs

```
from week12_algorithms import binary_search
# Tiny tests make equality and empty-interval behavior explicit.
assert binary_search([], 3) is None
assert binary_search([3], 3) == 0
assert binary_search([3], 4) is None
assert binary_search([1, 3, 5], 5) == 2
assert binary_search([1, 3, 5], 4) is None
print('Five bisection-search cases passed')
```

Add two more

Check a target smaller than the first element and greater than the last.

[W] Sections 10.9–10.12; [G] Section 10.1.2; teaching adaptation.

When the sortedness assumption is false

```
from week12_algorithms import binary_search, linear_search
# DELIBERATE PRECONDITION VIOLATION: values are not ascending.
values = [2, 100, 3, 4, 5]
print('Linear:', linear_search(values, 100))
print('Binary on unsorted input:', binary_search(values, 100))
```

Why 100 is missed

The midpoint value 3 directs the search right, discarding index 1 where 100 is stored.

[W] Sections 10.9–10.12; [G] Section 10.1.2; teaching adaptation.

NAZARBAYEV UNIVERSITY

Duplicate values reveal the output contract

keys	1	2	2	2	4

Operation	Result	Meaning
Manual binary search for 2	2	one valid matching index
First-match linear search	1	first position in this list
Leftmost exact search	1	first matching sorted position

Tests must match the promise

An any-match result should be tested for bounds and equality, not for an arbitrary preferred duplicate index.

[W] Sections 10.9–10.12; [G] Section 10.1.2; teaching adaptation.

NAZARBAYEV UNIVERSITY

bisect_left finds a position, not a match

```
# bisect_left gives an insertion position, even for
    an absent target.
values = [0.0, 0.5, 1.0, 1.5, 2.0, 2.5, 3.0]
print('Insertion position for 1.7:', bisect_left(
    values, 1.7))
print('Exact match for 1.7:', leftmost_match(values,
     1.7))
print('Position beyond last:', bisect_left(values,
    4.0))
```

Position for 1.7: 4

All values before position 4 are smaller; values from 4 onward are at least 1.7.

An endpoint is valid

The returned position may equal `len(values)`. Do not index it blindly.

[B] Python bisect documentation; [W] Section 10.12.

Build a leftmost exact-match wrapper

```
def leftmost_match(values, target):
    """FIRST exact match in an ascending sequence, or None."""
    position = bisect_left(values, target)
    # An insertion position can equal len(values) or point to another value.
    if position < len(values) and values[position] == target:
        return position
    return None
```

Two checks before returning a found index

The position must be in range, and the stored value must equal the target.

[B] Python bisect documentation; [W] Section 10.12.

Find the full range of equal values

```
from bisect import bisect_left, bisect_right
# Equal keys occupy one contiguous range in sorted data.
values = [1, 2, 2, 2, 4]
left = bisect_left(values, 2)
right = bisect_right(values, 2)
print(left, right, values[left:right])
```

Result

`1 4 [2, 2, 2]`. Count is `right - left`.
Locating bounds costs $O(\log n)$; copying k results also costs $O(k)$.

[B] Python bisect documentation; [W] Section 10.12.

NAZARBAYEV UNIVERSITY

Locate a calibration interval instead

Voltage (V)	0.0	0.5	1.0	1.5	2.0	2.5	3.0
Pressure (kPa)	0	50	100	150	200	250	300

Query: 1.7 V

It lies between 1.5 V and 2.0 V, so the interval has endpoint indices 3 and 4.

Different tasks

Interval lookup locates supporting data. Interpolation would compute an estimated pressure; it is not performed here.

[W] Sections 10.11–10.12; query and calibration values retained.

NAZARBAYEV UNIVERSITY

Specify the boundary ownership rule

Our explicit convention

$v_i \le x < v_{i+1}$, except that the final interval also includes the last grid value.

Examples

$x = 1.5$: interval 3 (right side).
$x = 3.0$: final interval 5.
Outside $[0, 3]$: `None`.

Prepare once

Require at least two finite, strictly increasing grid values. Freeze the validated grid before repeated queries.

[W] Section 10.11; boundary convention is an explicit teaching adaptation.

Interval lookup after preparation

```
def calibration_interval(grid, x):
    """Interval i in a prepared grid, or None outside; no extrapolation.
    grid[i] <= x < grid[i+1]; the final interval includes the upper endpoint.
    grid must satisfy prepare_grid's contract. Query validation is constant work.
    """
    if isinstance(x, bool) or not isinstance(x, (int, float)):
        raise TypeError("query must be a numeric voltage")
    if not isfinite(x):
        raise ValueError("query voltage must be finite")
    if x < grid[0] or x > grid[-1]:
        return None
    i = bisect_right(grid, x) - 1
    # An exact last-grid value belongs to the last available interval.
    return min(i, len(grid) - 2)
```

[W] Sections 10.11–10.12; [B]; explicit validated-interval adaptation.

NAZARBAYEV UNIVERSITY

A sorted index is not the original index

```
raw = [30, 10, 20]
indexed = sorted((value, index) for index, value in
    enumerate(raw))
keys = [pair[0] for pair in indexed]
p = bisect_left(keys, 20)
if p < len(keys) and keys[p] == 20:
    print('Sorted position:', p)
    print('Original index:', indexed[p][1])
print('Raw list:', raw)
```

Separate two coordinate systems

Raw: [30, 10, 20].
Sorted position of 20: 1.
Original index of 20: 2.

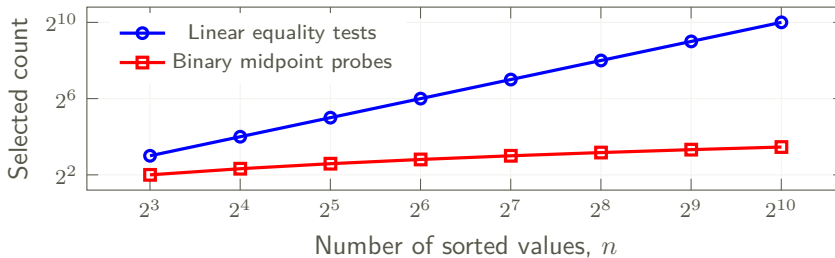
Prepare an indexed view

Retain original indices or timestamps when reordering data for repeated lookup.

[W] Appendix F; original example connecting Week 5 indices to ordered lookup.

NAZARBAYEV UNIVERSITY

Count probes on the same absent-target task



Counts from executed code; absent target is greater than all stored values. No wall-clock timing.

[W] Section 10.10; data generated from the supplied Week 12 counted implementations.

NAZARBAYEV UNIVERSITY

Include the cost of preparing the data

For q exact membership queries on n initially unsorted values:

Plan	Preparation	Queries
Repeated scans	none	$O(qn)$ worst case
Sort once, then bisect	$O(n \log n)$	$O(q \log n)$
Build an ID dictionary	$O(n)$ expected	$O(q)$ expected

Do not optimize a different task

Dictionary lookup needs a duplicate policy and does not replace ordered interval search.

[W] Sections 10.28–10.29 and Appendix G.16; [B] performance notes.

NAZARBAYEV UNIVERSITY

Activity: three outputs from the same sorted list

```
# Predict each result and justify every endpoint.
values = [2, 4, 4, 4, 8]
a = binary_search(values, 4)
b = leftmost_match(values, 4)
c = lower_bound(values, 5)  # First position with value >= 5.
```

1. Which result is a stored match?
2. Which result is only an insertion position?
3. What changes for an empty list?

Pairs - 4 min

[B] Python bisect documentation; [W] Section 10.12.

NAZARBAYEV UNIVERSITY

Debugging checklist for ordered search

1. Is the list ascending by exactly the searched key?
2. Do low/high use one consistent endpoint convention?
3. Does each unsuccessful iteration exclude a tested candidate?
4. Is absence handled without an invalid index access?
5. Are duplicate and tolerance policies explicit?
6. Are returned positions tied to the correct data ordering?

[W] Sections 10.9–10.12; [G] Section 10.1.2; teaching adaptation.

Lecture 35 exit ticket

Trace without running

Values `[3, 7, 9, 12]`, target 8. Use inclusive `low=0`, `high=3`.

1. List the visited midpoint indices.
2. What exact-match result is returned?
3. What insertion position would `bisect_left` return?
4. Name one reason that sorting before one search may be a poor choice.

Individual - 3 min

[W] Sections 10.9–10.12; [G] Section 10.1.2; teaching adaptation.

NAZARBAYEV UNIVERSITY

LECTURE 36

Sorting Algorithms and Engineering Rankings

Reorder complete records, compare algorithm mechanics, and verify the result

NAZARBAYEV UNIVERSITY

Lecture 36 learning goals

1. Specify sorting order, multiplicity, mutation, and tie behavior.
2. Use `sorted`, `sort`, and explicit key functions.
3. Trace selection sort and a stable merge-sort implementation.
4. Explain the work and extra storage of each implementation.
5. Filter engineering constraints before applying a ranking rule.
6. Verify sorted results without losing identities or raw data.

[W] Sections 10.13–10.19 and 10.30; teaching adaptation.

NAZARBAYEV UNIVERSITY

Warm-up: ordered numbers, wrong record

```
# Two aligned arrays initially describe real records.
names = ["steel", "aluminum", "copper"]
density = [7850, 2700, 8960]  # kg/m^3
density.sort()  # Names were not reordered.
print(names[0], density[0])
```

What went wrong?

The output associates steel with the density of aluminum. Sorting one field broke the record.

Predict and repair the representation - 3 min

[W] Sections 10.13–10.19 and 10.30; teaching adaptation.

NAZARBAYEV UNIVERSITY

A correct sort has several obligations

1. Return values in the specified order.
2. Preserve every element and its number of occurrences.
3. Keep each record's related fields attached.
4. Respect the promised input-mutation behavior.
5. Preserve equal-key order when stability is promised.

A sorted-looking list is not enough

For input `[3,1,3]`, output `[1,3]` is ordered but wrong: a duplicate vanished.

[W] Sections 10.13–10.19 and 10.30; teaching adaptation.

Use Python sorting for application code

```
# A new list; acquisition order is preserved.
raw = [800, 500, 1200]
ranked = sorted(raw)
# Explicitly mutate raw; return value is None.
result = raw.sort()
# Reverse the specified ordering.
high_first = sorted(raw, reverse=True)
```

Ask both questions

1. Was an existing list changed?
2. What value was returned?

No deep copy promised

A new list still refers to the same contained records.

[P] Python Sorting Techniques; [W] Sections 10.13–10.16.

NAZARBAYEV UNIVERSITY

A key function makes the criterion visible

```
# A record stays intact; only its key is compared.
def by_cost(record):
    return record["cost_USD"]
ranked = sorted(records, key=by_cost)
```

Meaning

Lower cost appears first; this does not establish feasibility or universal quality.

Execution

Pass the function, not `by_cost()`. Python computes one key per input record for this sort.

[P] Python Sorting Techniques; [W] Sections 10.13–10.16.

NAZARBAYEV UNIVERSITY

Stable sorting preserves equal-key order

arrival order	A:100	B:100	C:90
by cost	C:90	A:100	B:100

Stability

A remains before B because their cost keys are equal. With `reverse=True`, their relative order is still retained.

[P] Python Sorting Techniques; [W] Sections 10.13–10.16.

NAZARBAYEV UNIVERSITY

Several priorities fit in a tuple key

```
def design_key(record):
    """Lower cost first; use lower mass to resolve cost ties."""
    return record["cost_USD"], record["mass_kg"]
```

Design	Cost (USD)	Mass (kg)
A	100	4.0
B	100	3.0
C	90	5.0

Order under this rule

C, B, A. Cost is primary; mass breaks cost ties. Equal keys retain input order.

[P] Python Sorting Techniques; [W] Sections 10.13–10.16.

NAZARBAYEV UNIVERSITY

Selection sort: fix one position per pass

initial	7	3	5	2
pass 1	2	3	5	7
pass 2	2	3	5	7
pass 3	2	3	5	7

Rule

Find the smallest remaining value and swap it into the next unfinished position.

[W] Sections 10.13–10.19 and 10.30; teaching adaptation.

NAZARBAYEV UNIVERSITY

Selection sort: readable complete implementation

```
def selection_sort(values):
    """New sorted list; finite, comparable values; input remains unchanged."""
    result = values.copy()  # Copy references; preserve the caller's list.
    for i in range(len(result) - 1):
        smallest = i
        for j in range(i + 1, len(result)):
            if result[j] < result[smallest]:
                smallest = j
        # Extend the final sorted prefix by one smallest remaining value.
        result[i], result[smallest] = result[smallest], result[i]
    return result
```

Declared behavior

Return a new sorted outer list. The copied list is rearranged; the caller's list is preserved.

[W] Section 10.18; loop shortened to omit the unnecessary final pass.

Why selection sort is quadratic

At pass i, the algorithm compares $n - i - 1$ remaining values:

$$C(n) = (n-1) + (n-2) + \cdots + 1 = \frac{n(n-1)}{2}.$$

Even when already sorted

The minimum-search comparisons still occur. Best and worst time are $\Theta(n^2)$ in this implementation.

Memory accounting

The sorting loop uses constant auxiliary state; the preserving wrapper creates a linear-size copy.

[W] Sections 10.13–10.19 and 10.30; teaching adaptation.

NAZARBAYEV UNIVERSITY

A swap can break equal-key stability

initial	A:2	B:2	C:1
after first swap	C:1	B:2	A:2

Compare by numeric key only

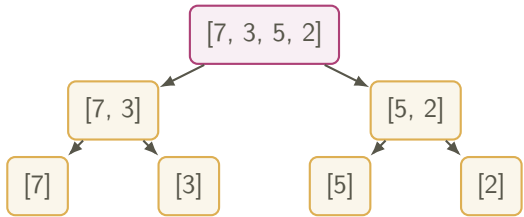
The first swap moves A past B. The usual swap-based selection sort is not stable.

Important distinction

The numeric values are sorted correctly, but an additional stable-record contract would not be satisfied.

[W] Sections 10.16 and 10.18; original stability counterexample.

Merge sort: divide, solve, and combine



Base case

A list of length zero or one is already sorted. Merge sorted sub-results on the return path.

[G] Sections 10.2–10.2.1; stable-merge teaching adaptation.

NAZARBAYEV UNIVERSITY

Merging two sorted lists uses two cursors

Left remainder	Right remainder	Output after next choice
[2, 7]	[3, 5]	[2]
[7]	[3, 5]	[2, 3]
[7]	[5]	[2, 3, 5]
[7]	[]	[2, 3, 5, 7]

Why the smallest front is enough

Each input is already sorted. No hidden smaller value exists later in that same input.

[G] Sections 10.2–10.2.1; stable-merge teaching adaptation.

NAZARBAYEV UNIVERSITY

A stable merge preserves both tails

```
def merge(left, right):
    """Merge two ascending lists into a new stable list; preserve inputs."""
    result = []
    i = j = 0
    while i < len(left) and j < len(right):
        # Prefer LEFT on a tie, preserving relative order of equal keys.
        if right[j] < left[i]:
            result.append(right[j])
            j += 1
        else:
            result.append(left[i])
            i += 1
    # Exactly one or both tails remain; each tail is already sorted.
    result.extend(left[i:])
    result.extend(right[j:])
    return result
```

[G] Sections 10.2–10.2.1; stable-merge teaching adaptation.

NAZARBAYEV UNIVERSITY

The recursive sorting function is short

```
def merge_sort(values):
    """New stable sorted list; ordinary comparisons; depth-first recursion."""
    if len(values) <= 1:
        return values.copy()  # Even the base case returns a new outer list.
    middle = len(values) // 2
    left = merge_sort(values[:middle])
    right = merge_sort(values[middle:])
    # Each half is sorted before it is merged with the other half.
    return merge(left, right)
```

Work on the return path

Each half is sorted before `merge(left, right)` is called. The base case also returns a new outer list.

[G] Sections 10.2–10.2.1; stable-merge teaching adaptation.

NAZARBAYEV UNIVERSITY

Derive time and peak space separately

Time

$T(n) = 2T(n/2) + \Theta(n)$.
$\Theta(n)$ work per level and $\Theta(\log n)$ levels give $\Theta(n \log n)$.

Peak extra storage

This depth-first implementation uses $O(n)$ list storage plus $O(\log n)$ active call frames.

Do not confuse totals

Cumulative copying across levels is $\Theta(n \log n)$; it is not all simultaneously live memory.

[G] Sections 10.2–10.2.1; stable-merge teaching adaptation.

NAZARBAYEV UNIVERSITY

Compare algorithms under stated contracts

Method	Time	Stable?	Our function preserves input?
Selection sort	$\Theta(n^2)$	No	Yes: makes copy
Insertion sort (optional)	$\Theta(n^2)$ worst	Yes, strict shifts	Yes: makes copy
Merge sort	$\Theta(n \log n)$	Yes, left ties	Yes: new lists
Python built-in sort	$O(n \log n)$ worst	Yes	`sorted`: yes; `sort`: no

Learning versus application

Implement small versions to understand the mechanism. Use documented built-ins for ordinary project sorting.

[W] Sections 10.17–10.19 and Appendix F.29; [G] Section 10.2; [P].

NAZARBAYEV UNIVERSITY

Test more than whether the output is ordered

```
from week12_algorithms import selection_sort, insertion_sort, merge_sort
# Independent expected outputs and non-mutation checks.
cases = [([], []), ([4], [4]), ([3, 1, 3, 2], [1, 2, 3, 3])]
for sorter in [selection_sort, insertion_sort, merge_sort]:
    for raw, expected in cases:
        before = raw.copy()
        result = sorter(raw)
        assert result == expected
        assert raw == before
        assert result is not raw
print('Nine sorter/case checks passed')
```

[W] Appendix F.39; Week 7 test design applied to sorting.

NAZARBAYEV UNIVERSITY

Engineering ranking: separate constraints and objectives

Design	Mass (kg)	Cost (USD)	Stress (MPa)	Strength (MPa)
A	3.5	180	140	250
B	4.2	160	100	250
C	3.8	210	90	250
D	3.9	190	80	200
E	2.9	195	110	300

Stated teaching requirements

FoS ≥ 2, mass ≤ 4 kg, cost ≤ 200 USD. Rank feasible designs by lower cost, then lower mass.

[W] Section 10.30; original dataset and criteria retained.

NAZARBAYEV UNIVERSITY

Preserve raw records while adding computed evidence

```
def rank_designs(records, required_fos=2.0, max_mass_kg=4.0, max_cost_usd=200):
    """Filter then rank teaching records; preserve caller records and order.
    Assumes finite numeric fields, stress_MPa > 0 and nonnegative costs/masses.
    New dictionaries hold computed FoS; sorted() alone would not copy records.
    """
    feasible = []
    for record in records:
        fos = record["yield_MPa"] / record["stress_MPa"]
        if (fos >= required_fos and record["mass_kg"] <= max_mass_kg
                and record["cost_USD"] <= max_cost_usd):
            enriched = record.copy()  # Do not add fields to the raw record.
            enriched["fos"] = fos
            feasible.append(enriched)
    return sorted(feasible, key=design_key)
```

[W] Section 10.30; explicit non-mutation adaptation.

Interpret the ranking, not just the first name

```
D: $190, 3.9 kg, FoS 2.50
E: $195, 2.9 kg, FoS 2.73
Recommended under stated rule: D
Raw records still lack a fos field: True
```

What the result supports

D is cheaper among the two feasible designs under the stated rule.

What it does not establish

Global engineering superiority, fatigue life, availability, environmental impact, or uncertainty bounds.

[W] Section 10.30; computed output checked against the supplied data.

NAZARBAYEV UNIVERSITY

Only one best record? A full sort may be unnecessary

```
# feasible is already a validated list of acceptable records.
if feasible:
    best = min(feasible, key=design_key)
else:
    best = None
```

Choose by the output

One optimum: a running minimum or `min`.
Full ranking: `sorted`.

Cost for k feasible records

One minimum: $O(k)$.
Full comparison sort: $O(k \log k)$.

[W] Section 10.7 and Appendix F.40–F.41; [P].

NAZARBAYEV UNIVERSITY

Activity: audit a plausible-looking sorting result

```
# INTENTIONAL CONTRACT DEFECT: duplicate records disappear.
def broken_sort(values):
    result = []
    for value in sorted(values):
        if value not in result:
            result.append(value)
    return result
```

Write a failing test first

Use the smallest duplicate-containing input. Which sorting obligation fails? What is the simplest repair?

Pairs - 3 min

Original teaching exercise within Session 12; assumptions stated on slide.

NAZARBAYEV UNIVERSITY

Assignment 7: preparation, not a new brief

Prepare to explain

- Search contract and chosen method.
- Sortedness or candidate-space assumptions.
- Sorting key, duplicates, and mutation.
- Tests and measured/derived work.

Before assessed submission

- Use the final Moodle statement.
- Check permitted tools and collaboration.
- Document and verify any assistance.
- Explain every returned result.

Scope

Session 12 lists Assignment 7. These slides introduce no deadline, weighting, or additional submission requirement.

[C] Course Specification, Session 12 (CLOs 1, 2).

NAZARBAYEV UNIVERSITY

Lecture 36 exit ticket

1. Why can an ordered result still be an incorrect sort?
2. For `[4,1,3,1]`, trace the first selection-sort pass.
3. Why must merge preserve the unconsumed tail?
4. Which tie rule makes this merge implementation stable?
5. Choose a tool for one cheapest feasible record versus a complete ranking.

Individual - 3 min

Week 12 in one sentence

Choose a method whose output, ordering assumptions, and data-preservation behavior match the engineering question.

[W] Sections 10.13–10.19 and 10.30; teaching adaptation.

NAZARBAYEV UNIVERSITY

Optional: insertion sort builds a sorted prefix

```
def insertion_sort(values):
    """New stable sorted list; shift only STRICTLY larger preceding values."""
    result = values.copy()
    for i in range(1, len(result)):
        current = result[i]  # Hold it while making room in the sorted prefix.
        j = i - 1
        while j >= 0 and current < result[j]:
            result[j + 1] = result[j]
            j -= 1
        result[j + 1] = current
    return result
```

Different invariant

The processed prefix is sorted, but not necessarily in its final global positions until all values are inserted.

[W] Section 10.19; optional implementation alternative.

NAZARBAYEV UNIVERSITY

Optional: nearly sorted input changes insertion work

Input pattern	Element shifts	Time for this implementation
Already ascending	0	$\Theta(n)$
Strictly descending	$n(n-1)/2$	$\Theta(n^2)$
Few out-of-order pairs	Fewer shifts than reversed input	Depends on those inversions

Key comparison versus loop condition

Only preceding data comparisons count as key comparisons; tests such as `j >= 0` are separate operations.

[W] Section 10.19; original count interpretation.

NAZARBAYEV UNIVERSITY

Optional: a manual lower-bound search

```
def lower_bound(values, target):
    """First position with value >= target; len(values) if all are smaller."""
    low, high = 0, len(values)
    # Search positions using a half-open candidate interval [low, high).
    while low < high:
        mid = (low + high) // 2
        if values[mid] < target:
            low = mid + 1
        else:
            high = mid  # This midpoint may still be the first suitable one.
    return low
```

A different endpoint convention

`high = len(values)` and `while low < high`.
Do not splice these lines into the inclusive exact search without changing the invariant.

[B] bisect_left contract; original manual half-open implementation.

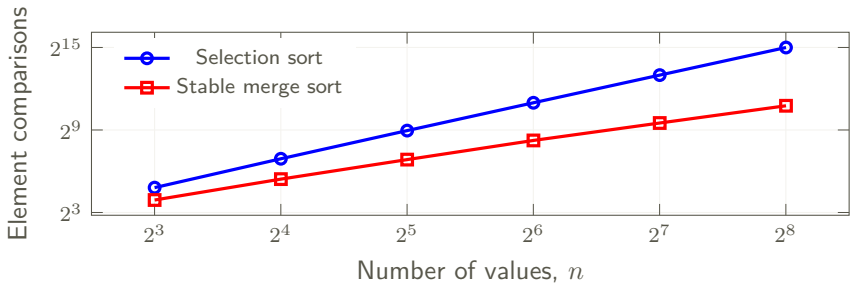
NAZARBAYEV UNIVERSITY

Optional: sort complete objects by a measurement

```
# Optional link to Week 9: sort complete instances using a named key.
class Reading:
    def __init__(self, name, pressure_kpa):
        self.name = name
        self.pressure_kpa = pressure_kpa
def pressure_key(reading):
    return reading.pressure_kpa
readings = [Reading('P-01', 120), Reading('P-02', 90)]
for reading in sorted(readings, key=pressure_key):
    print(reading.name, reading.pressure_kpa, 'kPa')
```

[W] Appendix F; [P] key functions; original class example.

Optional: measured comparisons are not timing



Executed on the same seeded shuffled list at each n; selections are exact counts of data comparisons.

Executed Week 12 counters; [W] Sections 10.18–10.20; [G] Section 10.2.1.

NAZARBAYEV UNIVERSITY

Searching and sorting: method-selection reference

Need	Starting method	Critical condition
One first event in time order	Linear predicate search	Do not reorder chronology
Any exact hit in sorted data	Binary search	Ascending consistent keys
First equal key / insertion	`bisect_left`	Check equality for exact hits
One best feasible record	Filter + minimum	Declare constraints and tie rule
Full stable ranking	`sorted(..., key=...)`	Keep complete records together
Small candidate combinations	Brute-force enumeration	Count and cover every candidate

[W] Appendix F.40–F.41; synthesis of the week.

Source basis and adaptation record

Supplied course sources

[C] ENG 101 Course Specification Report, Session 12 (CLOs 1, 2).
[W] Y. Wang, *Programming for Engineers with Python*, supplied draft v20260815, Chapter 10 and Appendix F.
[G] J. V. Guttag, *Introduction to Computation and Programming Using Python*, 2nd ed., Sections 10.1–10.2.

Official Python checks

[B] bisect documentation. [P] Sorting Techniques.

Examples are teaching adaptations or labelled original exercises. The source map documents contract changes; no assessment requirements are inferred.

Source details and per-example adaptations are included in SOURCE_MAP.md.

Connect Week 12 to the remaining course

What now transfers

- An algorithm has a language-independent contract.
- Correctness arguments survive translation.
- Indices, mutation, and libraries may differ.

Week 13 and Week 14

- Compare Python with other languages.
- Explain equivalent algorithms in different syntax.
- Review the complete engineering-computation workflow.

Preparation habit

Predict, trace, implement, test, and explain the result before replacing the implementation with a library call.

[C] Sessions 12–14; curriculum synthesis.

Week 12: the engineering habit to keep

A faster answer is useful only when it answers the right question.

Before trusting a search or ranking

State the contract. Check the assumptions. Preserve the record. Test the boundaries. Explain the result.